# Implementation plan

# 1 Project description

## 1.1 Name of the project

**Smart Desk Guardian (SDG)**: an IoT device that monitors workspace ergonomics and wellness.

## 1.2 Goal of the project

SDG helps reduce digital eye strain, sitting too close to the screen and long sitting periods for computer users. An Arduino measures screen distance, seat occupancy, ambient light and heart rate. A Raspberry Pi warns the user on the spot with an OLED message and a buzzer when a threshold is crossed: closer than 50 cm to the screen, seated for more than 45 minutes, a room too dark for screen work, or a heart rate above the user's resting baseline during an ECG check.

The Pi also publishes all readings over MQTT to a custom app. The app shows live values, alerts and session history. Local warnings work even when the app or network is offline.

Data flow: sensors → Arduino Uno (reads and filters) → USB serial, JSON → Raspberry Pi (alert logic, OLED, buzzer, Mosquitto MQTT broker) → MQTT over WebSockets → custom app.

## 1.3 Hardware used in the project

Arduino Uno R3 + Gravity shield, raspberry Pi, ultrasonic sensor, pressure sensor, ecg heart rate sensor, ambient light sensor, oled display, buzzer and cables.

# 2 Schedule

## 2.1 How the project is progressing

The project runs 61 days (about 9 weeks), with a teacher review meeting every week. In Week 1 the team agrees on the MQTT topics and JSON format, so the Arduino, Pi and app can be built in parallel. Each part is tested on its own first, then integrated end to end in Week 7. Feature freeze is at the end of Week 8.

## 2.2 Milestones

| Week | Milestone | Teacher review |
|---|---|---|
| 1 | Plan approved; lab inventory checked (ECG, light sensor, FSR); Git repo; MQTT topics and JSON format agreed | Plan approval |
| 2 | Ultrasonic and FSR wired and calibrated on the Arduino; app wireframes | Serial monitor demo |
| 3 | ECG and light sensors added; Arduino sends all readings as JSON; app skeleton | Complere sensor node |
| 4 | Arduino to Pi serial link; Mosquitto broker running; Python bridge publishes readings | Messages visible in an MQTT client |
| 5 | OLED and buzzer alert logic on the Pi; app shows live values | Local alerts demo |
| 6 | App alerts screen and session history | App demo with live data |
| 7 | End-to-end integration; false-alarm filtering | Full system demo |
| 8 | Enclosure, stability test, bug fixes; feature freeze | Prototype inspection |
| 9 | Slides, backup demo video, final demo | Final grading and seminar |

# 3 Resources

## 3.1 Who does what?

The project is budgeted at 120 hours, 40 per person (about 4.5 hours a week each). At a student engineering rate of €20/hour, that is €2,400 of labor.

| Task | Lead | Hours |
|---|---|---|
| Project management, weekly reports, documentation | Mehdi Tahari | 10 |
| Arduino firmware, sensor wiring and calibration | Mehdi Tahari | 18 |
| Raspberry Pi setup and serial-to-MQTT bridge (Python) | Angel Velazquez | 10 |
| Mosquitto broker, MQTT topics, credentials and connection reliability | Angel Velazquez | 10 |
| OLED display and buzzer alert logic | Angel Velazquez | 8 |
| App UI design (screens, alert flow) | Adam Azizi | 8 |
| App development: live dashboard, alerts, history | Adam Azizi | 20 |
| Integration, testing, enclosure and seminar | All three | 36 |
| Total | | 120 |

## 3.2 Who is the project manager?

Mehdi Tahari

## 3.3 What other roles are in the project?

• Angel Velazquez, IoT gateway engineer: Raspberry Pi, Python serial bridge, MQTT broker, OLED and buzzer.

• Adam Azizi, app developer: design and build of the custom app that shows live data, alerts and history.

# 4 Risks

## 4.1 If there are uncertainties at the moment?

We have uncertainty of the availability of the ecg hearth sensor.
We are still doubting what to use for the Dashboard app, either ThingSpeak or a custom React Native app.